

ZKPoG: Accelerating WitGen-Incorporated End-to-End Zero-Knowledge Proof on GPU

Muyang Li
Tsinghua University

Yueteng Yu
Tsinghua University

Bangyan Wang

Xiong Fan
Cysic Inc.

Shuwen Deng
Tsinghua University

Abstract

Zero-Knowledge Proof (ZKP) is a cornerstone technology in privacy-preserving computing, addressing critical challenges in domains such as finance and healthcare by ensuring data confidentiality during computation. However, the high computational overhead of ZKP, particularly in proof generation and verification, limits its scalability and usability in real-world applications. Existing efforts to accelerate ZKP primarily focus on specific components, such as polynomial commitment schemes or elliptic curve operations, but fail to deliver an integrated, flexible, and efficient end-to-end solution that includes witness generation.

In this work, we present ZKPoG, a GPU-based ZKP acceleration platform that achieves full end-to-end optimization. ZKPoG addresses three key challenges: (1) designing a witness-generation-incorporated flow for Plonkish circuits, enabling seamless integration of frontend and backend with GPU acceleration; (2) optimizing memory usage to accommodate large-scale circuits on affordable GPUs with limited memory; and (3) introducing an automated compiler for custom gates, simplifying adaptation to diverse applications. Experimental results on an NVIDIA RTX 4090 GPU show on average $22.8\times$ end-to-end acceleration compared to state-of-the-art CPU implementations and on average $12.7\times$ speedup over existing GPU-based approaches.

1 Introduction

Zero-knowledge proof (ZKP) serves as a foundational technology in privacy-preserving computing [4, 32], addressing privacy challenges in domains such as finance [27] and healthcare [22]. Privacy-preserving computing aims at securing data in use, when it is being processed in memory by hardware or software systems. At the core of many privacy-preserving computing frameworks is ZKP, a cryptographic scheme that enables one party (the prover) to demonstrate the truth of a statement to another party (the verifier) without disclosing any additional information [15]. The versatility of ZKP extends

across various applications. In blockchain systems, ZKP has revolutionized transaction confidentiality, enabling protocols like Zcash to achieve private transfers without compromising auditability [36]. A notable example is zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge), which allow for efficient verification of transactions with minimal computational overhead, scaling to support thousands of transactions per second [6]. In authentication systems, ZKP reduces the dependence on traditional passwords by enabling identity verification without exposing sensitive user credentials [18].

Despite its transformative potential, ZKP faces significant computational challenges, particularly in supporting to large-scale applications. Generating and verifying proofs often involve complex computational operations, such as multi-scalar multiplications and number-theoretic transforms, which require substantial computational resources [6, 14]. This results in that, for instance, in blockchain systems, while zk-SNARKs have enabled private transactions, their implementation can result in proof generation times ranging from several seconds to minutes, limiting real-time scalability [36]. Similarly, deploying ZKP in federated learning or secure authentication systems frequently leads to increased latency and energy consumption [2, 24, 37, 37].

In a zk-SNARK, any problem is first expressed as a circuit¹ $C(\cdot)$ and a target output y ; Valid solutions are those assignments of circuit inputs that produce the target output, i.e., $\{x \mid C(x) = y\}$. Next, these circuits are compiled (or “encoded”) into a fixed form of problem: There exists a set of polynomials that satisfy certain polynomial equations². The rest is handled by a fixed zkSNARK system that can prove the satisfaction of these polynomial equations.

The prover’s workload cost consists of three major components:

¹The circuits mentioned here are not Boolean circuits but arithmetic circuits, where the values of wires are finite field elements. Boolean circuits are a special case, which can be encoded by representing $a \wedge b$ as $a \times b$ and $\neg a$ as $1 - a$.

²These polynomials are defined over a finite field.

- **Setup:** A one-time cost per circuit to translate the circuit into polynomial equations.
- **Witness Generation (Frontend):** For each new input, the cost to execute the circuit with the input and prepare the solution to the polynomial equations.
- **Backend:** The cost to prove the polynomial equations.

Existing works tend to speed up ZKP computation, but fall short of providing a flexible and efficient end-to-end acceleration platform. These works [10, 25, 29–31, 38, 43] demonstrate significant advances in specific ZKP components such as Multi-Scalar Multiplication (MSM) and Number Theoretic Transform (NTT), but neglect the absence of a unified, flexible real end-to-end platform capable of supporting comprehensive ZKP pipelines, including witness generation (WitGen), proof computation, and verification across diverse use cases.

1.1 Our Contributions

In this paper, we present ZKPoG, which achieves a GPU-based auto-compiled witness-generation-incorporated end-to-end zero-knowledge proof acceleration platform. We identify and address three critical challenges that hinder the development of ZKPoG and provide corresponding solutions:

End-to-End WitGen-Integrated Design. Achieving a seamless end-to-end design for ZKP acceleration, that integrates both the frontend with the backend, is an unexplored area. To address this, we optimize witness generation (WitGen), tackling irregular computation patterns and data dependencies. In particular, we propose a new method to analyze the circuit and sort gates within the setup stage for the parallelized execution of WitGen. All steps are fully accelerated on GPUs to achieve significant performance improvements.

Memory Optimization for Affordable Hardware. Real-world applications of ZKP often require processing tens of gigabytes of data, which presents a significant hurdle for accelerators with limited memory. By eliminating redundant buffers and optimizing data generation, ZKPoG reduces peak memory usage, enabling the processing of larger circuits on affordable hardware.

Auto-Compilations for Custom Gates. Custom gates, essential for optimizing application-specific computations in Plonkish circuits, are typically implemented using high-level programming languages like Rust [28, 42]. Translating these to GPU-friendly code is non-trivial due to limited programmability and hardware constraints. In this case, we develop an automated compiler that generates optimized GPU and CPU code for custom gates from minimal user-defined scripts. This lowers the barrier to adapting ZKPoG for new applications, facilitating rapid deployment in evolving ZKP protocols.

We implement and test ZKPoG on NVIDIA RTX 4090 GPU, and achieve on average $22.8\times$ end-to-end acceleration ratio on real-world scenarios compared to the state-of-the-art parallel CPU implementations, and over $37\times$ speedups excluding the witness generation step. Compared to an existing

Name	Platform	Arithmetization	PCS	E2E
BachZK [29]	GPU	R1CS	Hash	●
NoCap [38]	ASIC	R1CS	Hash	●
SZKP [10]	ASIC	R1CS	EC	●
DistMSM [25]	GPU	R1CS	EC	○
CuZK [30]	GPU	R1CS	EC	○
GZKP [31]	GPU	R1CS	EC	○
PipeZK [43]	ASIC	R1CS	EC	○
ZKPoG	GPU	Plonkish	Hash	●

Table 1: Existing work does not support Plonkish arithmetization. They also do not achieve full end-to-end acceleration of proof generation.

open-source GPU implementation with only a fraction of the steps accelerated, we achieve on average an additional $12.7\times$ speedups.

1.2 Related Work

The major obstacle to the widespread adoption of zk-SNARK is the computational overhead on the prover side, requiring the need for acceleration. According to our investigation in Table 1, most of existing work [25, 30, 31, 43] focuses on the acceleration of typical kernels such as NTT and MSM, while several recent works claim to achieve an end-to-end implementation [10, 29, 38].

For the works accelerating NTT and MSM, these kernels are indeed the bottleneck of zk-SNARK systems using them as Polynomial Commitment Scheme (PCS). However, in modern systems using hash-based PCS, the polynomial commitment step is faster and no longer the dominant part of prover time. As shown in Figure 1 and Figure 2, time distribution of the prover is very scattered, and polynomial commitments (the sum of NTT, Reverse and Merkle tree) only take up less than half of the prover time. This means that the obtained acceleration ratio will be very limited if existing acceleration for NTT and MSM is applied directly, not to mention the time spent on data transfer between the CPU and the accelerator.

NoCap [38] is an ASIC accelerator for Spartan [40] IOP and Orion [41] PCS that claims to provide end-to-end acceleration. SZKP [10] is claimed to be the first ASIC framework that accelerates full zkSNARK proofs on-chip. However, what we must emphasize here is that these efforts do not consider the cost of witness generation. Since Witness cannot be reused when input changes, so witness generation will still put an upper bound on the proof generation speedup.

In summary, existing work does not treat the entire process (including witness generation) of proof generation as the acceleration target. The currently popular Plonkish Arithmetization that makes fast provers is not considered as well. Therefore, we want to propose a new ZKP-acceleration frame-

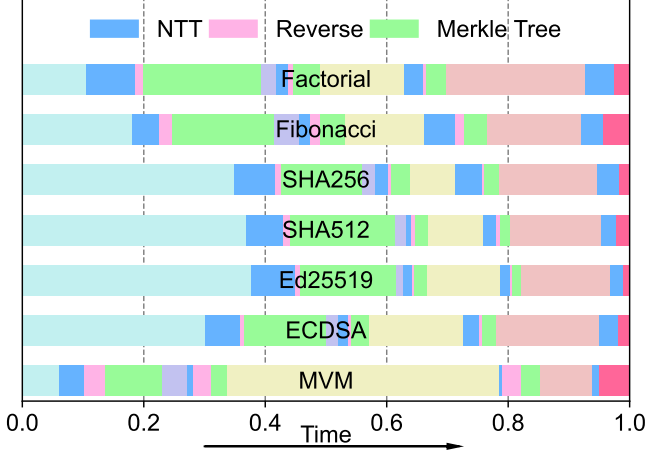


Figure 1: Time share of different steps as proof generation progresses. Unlabeled colors in the figure represent witness generation, partial product, quotient, final polynomial and FRI proof in sequence.

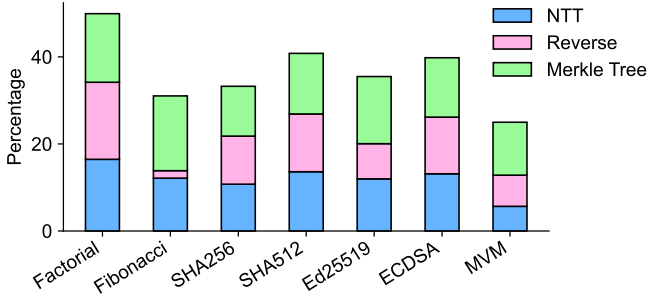


Figure 2: Time share of different kernels that are used in polynomial commitment in the whole proof generation. Accelerating them provides up to a 2x end-to-end speedup.

work, ZKPOG, that can accelerate the whole prove procedure from circuit input to final proof, and seamlessly adapt to custom gates on readily available devices.

2 Deep Dive Into ZK Proof Generation

In this section, we analyze the current state of zk-SNARK systems, identify optimal combinations of encoding and proving systems, examine challenges faced by emerging systems, and present our proposed solutions to address these challenges.

We first describe two components of the zk-SNARK system, which are the Plonkish arithmetization and polynomial commitment scheme. As two critical factors shaping the properties of a zk-SNARK system, different selections yield distinct advantages and trade-offs. Next we move on to provide an overview of NVIDIA GPU and then describe the challenges we faced and our solutions.

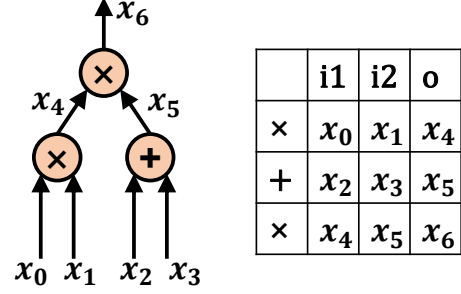


Figure 3: A simple circuit with Plonkish arithmetization. Witness matrix contains the values of gate ports.

Arithmetization	RICS	Plonkish
Witness Format	vector	matrix
Wire Encoding	sparse matrices	copy constraints
Gate Type	add / mul	custom
Number of Witness	more	less
Polynomial Degree	high	medium

Table 2: Comparison between RICS and Plonkish.

2.1 Plonkish Arithmetization

One of the early widely-used arithmetization techniques is the Rank-1 Constraint System (RICS). In such a system, all gates in the circuit have to be either (modular) addition ($a + b \bmod p$) or multiplication ($a \times b \bmod p$). The wire topology is encoded as sparse matrices A, B, C , where the sparse pattern encodes the wire topology in a fashion such that $A[i, j]$ is non-zero if and only if the j -th wire (net) is connected to the first input port of the i -th gate. B and C play similar roles for the second input and output ports of the gates. Readers may refer to [5, 16] for a more complete description.

Plonkish is a newer arithmetization technique that originated from Ariel Gabizon et al.’s famous work “Plonk” in 2019 [13]. People have found that one particular component of Plonk, i.e., the idea of using copy-constraints to express arbitrary wire topology, is particularly advantageous compared to earlier sparsity-pattern-based encoding. Since then, this technique has become the standard way to encode wire topology in both academic and industrial zk-SNARK systems [8, 12, 28, 42]; systems with this characteristic are often referred to as Plonk-ish systems. As shown in Figure 3, witness is a matrix containing values at I/O ports of each gate, and the prover needs to prove that the input and output fit the gate type, and that the wire topology is correct.

As shown in Table 2, compared with sparsity-pattern-based encoding like RICS, a uniquely advantageous feature of the Plonkish system is that the types of gates are not restricted to addition and multiplication but can also support arbitrarily complicated custom gates in the frontend. With the degree of freedom, first, one custom gate can often replace a large

PCS	EC-based (KZG)	hash-based (FRI)
Finite Field	elliptic curve ≥ 256 -bit	arbitrary (64-bit ok)
Operation	MSM, NTT	NTT, Hash, Merkle Tree
Commitment Size	small	medium
Verification Time	small	medium
Commitment Time	high	low
Proving Key Size	very large (KZG)	small
Trusted Setup-Free	✗ for KZG	✓
Post-Quantum Secure	✗	✓

Table 3: Comparison between EC-based and hash-based PCS.

Degree	KZG Time(s)	FRI Time(s)
2^{16}	8.535	0.091
2^{17}	15.094	0.158
2^{18}	25.425	0.324
2^{19}	45.192	0.747
2^{20}	80.920	1.519

Table 4: Time for committing 16 polynomials with KZG or FRI. The data is collected from Icycle [23] and Plonky2 [28].

number of addition and multiplication gates and extend the functionality — its benefits are similar to a custom instruction in a CPU. Second, if the circuit contains a repeated sub-circuit many times, defining a custom gate can significantly reduce the number of gates in the circuit, thus reducing the proof time by orders of magnitude, which is confirmed by our experiments.

In addition, the Plonkish system does not need to work with sparse matrices, which is hardware-friendly for acceleration. R1CS is advantageous only in certain special scenarios; for example, in the Groth16 [18] system, it is a good fit for the KZG commitment scheme on elliptic curves with pairing structures. Compared with the R1CS constraint system, the Plonkish system has been used by well established projects, including big Ethereum-layer 2 such as zkSync [44], Scroll [39] and others.

As shown in Table 5, we found that different applications use different gates, many of which are custom gates. Therefore, support for custom gates is important for a system that claims to accelerate Plonkish zkSNARK.

2.2 Polynomial Commitment Scheme

The standard way to prove that a set of polynomials can satisfy certain polynomial equations logically consists of two steps. First, we assume that there is a hypothetical third party called “the oracle” that both the prover and the verifier can trust.

The prover first sends the solution polynomials to the oracle; then the verifier randomly chooses one field element and asks the oracle to show the evaluation of the polynomials at that point. Mathematical properties ensure that checking at a random point is sufficient to guarantee that the whole set of polynomial equations are satisfied with overwhelmingly high probability.

Since the oracle cannot exist in reality, it is replaced by a polynomial commitment scheme (PCS) [26], which works as follows: 1) the action of “the prover sending a polynomial to the oracle” is replaced by “the prover computing a commitment (something like a hash fingerprint) of the polynomial” and directly sending it to the verifier; 2) the action of “the oracle showing the evaluation of the polynomial at a point” is replaced by “the prover sending the evaluation of the polynomial together with a magic object that can convince the verifier that the evaluation is consistent with the previous commitment.” This step is called “opening” the commitment.

Early PCS systems rely on the group structure of elliptic curves, in particular, the KZG commitment scheme [26]. KZG is the PCS that has the smallest commitment size and verification time. Its main computational workload is MSM and on the prover side. However, KZG is based on elliptic curve cryptography with expensive operations on the curve, resulting in a much higher prover time. Specifically, with KZG commitment scheme, the workload to commit one field element lead to an average of ~ 240 modular multiplications over the base field of the curve; each of which translates to *three* large integer multiplications (either 256 or 384 bits prime for common curves). Those finally add up to $\sim 11 \times 10^3$ 64-bit integer multiplications. In comparison, hash-based PCS only requires one hash function with input message shorter than 1024-bit to commit one field element. Depending on the choice of hash function, the cost is either to evaluate bunch of bitwise XOR (if using SHA2/3) or less than 10^3 64-bit integer multiplication (if using Poseidon hash function [17]). As shown in Table 4, this often results in a difference of more than one order of magnitude.

Finally, KZG requires a structured reference string (SRS) that involves a trusted setup process, leaving potential security risks. In addition, the SRS is a huge constant (several GBs) that has to be downloaded and stored by every prover using the system as the proving key. It also occupies precious memory, particularly on the GPU. The hash-based PCS does not have the above problems at all.

In addition, it restricts the choice of the underlying field to be used in the zk-SNARK system, usually requiring a large prime of at least 256 bits. This is not desirable from the perspective of performance. Besides, KZG is not post-quantum secure. The post-quantum security problem applies to all elliptic curve-based PCS systems. Last but not least, for hash-based PCS such as Fast Reed-Solomon Interactive oracle proof of proximity (FRI) [3], the rest of the prover outside the PCS can use a more hardware-friendly prime field,

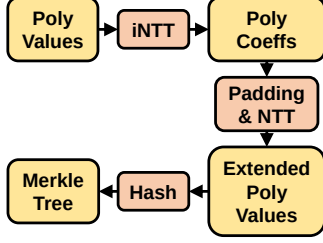


Figure 4: Workflow of FRI PCS.

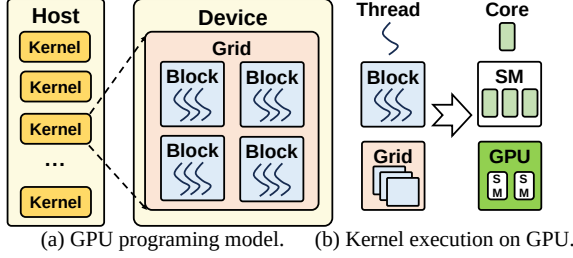


Figure 5: GPU programming model and the kernel execution on GPU.

such as the 64-bit Goldilocks field ($p = 2^{64} - 2^{32} + 1$), whose modular arithmetic is again more than an order of magnitude faster than the 256-bit prime field. The comparison between EC-based and Hash-based PCS is summarized in Table 3.

Therefore, in current practice, people turn to hash-based PCS like FRI [3], Orion [41] and Liger [1] for better prover performance. A complex circuit is often proved in a recursive manner [7], where hash-based PCS is used in the first few steps to generate a relatively long but easy proof (compared with KZG proof) and EC-based PCS is used in the last step to prove that the proof passes a relatively simple verification circuit (compared with the original circuit). This trade-off combines the advantages of both options, providing a fast prover and a fast verifier.

In the recursive proof scheme, hash-based PCS is used on much bigger circuits and becomes the bottleneck of proof generation. The performance of KZG improves with accelerated NTT and MSM, but for hash-based PCS, different kernels are to consider. As shown in Figure 4, the compression of polynomials is achieved by a hash function in FRI.

2.3 NVIDIA GPU Features

General Purpose Graphics Processing Units (GPUs) have become essential in machine learning (ML) systems due to their exceptional parallel processing capabilities. Popular programming models such as OpenCL [19], OpenGL [20], OpenACC [34], and CUDA [33] enable the development of high-performance GPU-accelerated ML applications. These models facilitate the efficient use of GPU resources for compute-intensive tasks, contributing to the widespread adoption of

GPUs across data-intensive domains.

Figure 5(b) provides an overview of GPU architecture, highlighting multiple Streaming Multiprocessors (SMs), each equipped with numerous cores to execute individual threads. Threads within a block run on the same SM, and an SM can handle multiple blocks simultaneously. This architecture supports Single Instruction Multiple Thread (SIMT) execution, allowing threads to operate concurrently under the same instruction, greatly enhancing processing efficiency.

GPU programming models like CUDA simplify the process of offloading tasks to GPUs by providing structured execution hierarchies. As shown in Figure 5(a), tasks are defined as kernel functions and executed by threads organized into blocks and grids. Built-in indexing variables, such as *threadIdx* and *blockIdx*, help manage data structures efficiently [21], enabling GPUs to achieve exceptional performance in parallel computations.

2.4 The Challenges Addressed by ZKPoG

In order to achieve such a ZKP acceleration design framework, a thorough analysis and acceleration of the whole proof generation are required. This acceleration may focus on the unstudied computations of Plonkish, and accommodate its flexibility.

WitGen-integrated end-to-end design. Achieving an end-to-end design for ZKP acceleration including the frontend (use Plonkish) and backend (use FRI) to balance the requirements in a unified framework, which remains underexplored in ZKP acceleration. Witness generation, a critical yet often neglected step, involves converting problem statements into constraints and satisfying them with valid inputs. This step demands significant memory and computational resources, and optimizing it for high-performance hardware like GPUs or FPGAs is particularly challenging due to data dependencies and irregular computation patterns.

Memory optimization. For many real-life applications, the generation of proof often involves tens of gigabytes of data. But memory is expensive for almost all sorts of accelerator. Memory optimization of the algorithm is not explored yet, making a concurrent work [29] use NVIDIA GH200 with 96GB device memory and \$40K price. In contrast, we wish to provide acceleration for more available platforms like NVIDIA RTX 4090 with 24GB device memory and \$2K price, which makes memory optimization even more important.

Compiler for custom gates. Custom gates are usually implemented based on a high-level programming language that supports generic programming like Rust [28, 42]. These languages can easily cope with variable-length parameters and overloading. For any computation outside CPU, this is not easy. Though some existing gates can be manually implemented, more potential gates will emerge with new applications in the ZKP domain. An interface for custom gates is to design for accelerators, and code should be automatically

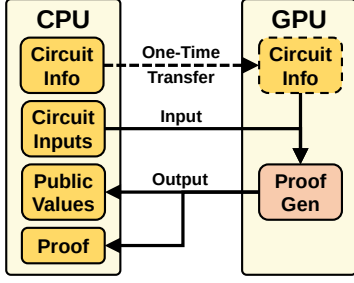


Figure 6: Interface of ZKPoG between CPU and GPU.

generated to make the best possible use of GPU programmability.

2.5 Our proposal

With all needs and challenges above in mind, we design ZK-POG, which is a GPU-based witgen-incorporated end-to-end zero-knowledge proof acceleration platform. We use GPU for its (re-)programmability for custom gates and accessibility for real-world users. This choice also adapts to the ever-changing ZKP protocols. We choose Plonky2 [28] as a standard and optimized CPU implementation of Plonkish arithmetization combined with FRI PCS, and also the baseline of acceleration.

Our proposal tackles the challenges illustrated in Section 2.4 using the following solutions with their corresponding advantages:

- *WitGen-Integrated end-to-end flow*: All steps in the proving stage are done and accelerated on GPU, resulting in an average speedup of $22.8\times$ on real-life applications. In particular, for witness generation we sort the gates in the setup stage and execute them in parallel.
- *Memory optimization*: The peak memory usage of GPU is optimized and bigger circuits are allowed to prove by reducing redundant data buffers and ill-timed data generation.
- *Compiler for custom gates*: The GPU and CPU code of a custom gate operations can be auto-compiled based on a minimal script of the gate provided by the programmer, thus greatly reducing the difficulty of adapting to new applications.

3 ZKPoG Framework

3.1 Overall Workflow

In this section, we demonstrate the overall workflow of our ZKPoG design and iterate how we do the WitGen-Integrated end-to-end flow and memory optimization to tackle the challenges.

As demonstrated in Figure 6, the CPU prepares data used in the proving stage and passes it to the GPU. Then GPU is

ready to prove and gets input of the circuit from CPU, after which GPU follows the workflow in Figure 7, and only the data used in the final proof goes back to CPU, where it is assembled into a proof structure for local and remote verifiers on CPU to verify.

Each of these steps is carefully analyzed and implemented on GPU to achieve the best end-to-end performance. To save space, we focus on the most important optimizations here.

3.2 Plonk Polynomial IOP

3.2.1 Witness Generation

Gate execution and data management for witness generation. The witness generation process requires the systematic execution of every gate within the circuit, inherently governed by the dependency hierarchy among the gates. Gates are executed iteratively, starting from an initial list, with dependent gates dynamically added to a secondary list. This iterative dependency resolution and execution cycle persist until the entire circuit is traversed and all gates are executed. The sequential nature of this process introduces significant computational bottlenecks, particularly for complex circuits with extensive inter-gate dependencies.

Figure 3 illustrates an example where the input of a top multiplication gate depends on the output of two lower-level gates. While the top gate executes later, the two lower gates have no dependencies between them and can execute in parallel. Leveraging this principle, we propose the GPU-accelerated approach that parallelizes the execution of independent gates, requiring GPUs to identify and schedule executable gates efficiently. Ensuring the correct execution order is critical, necessitating a robust gate sorting mechanism before execution.

Efficient gate sorting for parallel execution. The sorting process, while an additional step, is computationally inexpensive in scenarios where the circuit is predetermined by the application scenario and preprocessed in the setup stage. The dependency relationships between gates and their parameters (e.g., input targets in the witness matrix) remain fixed. Consequently, sorting is performed only once during the build stage, aligning with the common "build once, prove many times" workflow. This ensures that the sorting overhead is amortized across multiple proofs, rendering its impact negligible.

Hashmaps and hierarchical data organization. During sorting, gates are first ordered based on their dependencies and divided into levels. Gates within the same level are grouped by type and stored in a hashmap, where the key represents the gate type, and the value contains the parameters of all gates of that type. This organization results in a structured list of hashmaps, as illustrated in Figure 8(a).

Data transfer to GPU. To facilitate GPU execution, the sorted gate information is packaged for efficient transfer. The values of each hashmap entry are sequentially appended to a large array, while a lookup table is constructed to store

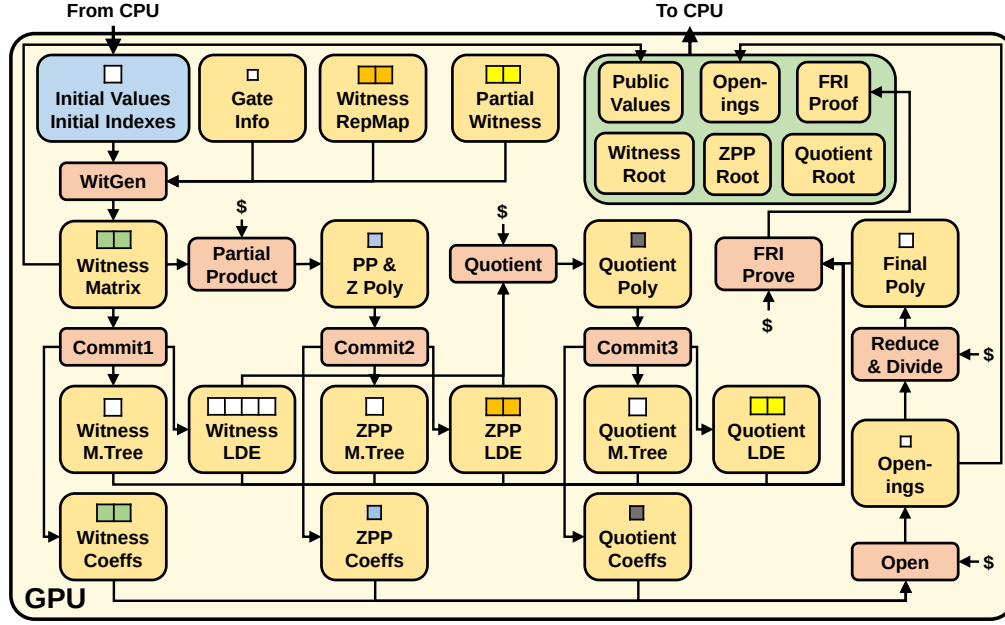


Figure 7: Overall proof generation workflow of ZKPoG, the size and number of squares reflect the GPU memory usage, and data that uses the same memory space in turn is represented by the same colored squares. Dollar symbol stands for random values from Fiat-Shamir Transform [11]. Some pre-calculated values are not shown.

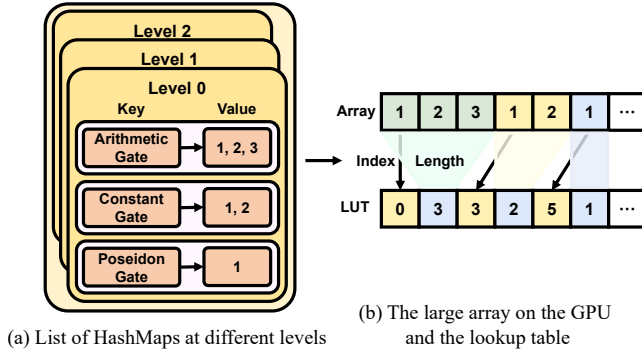
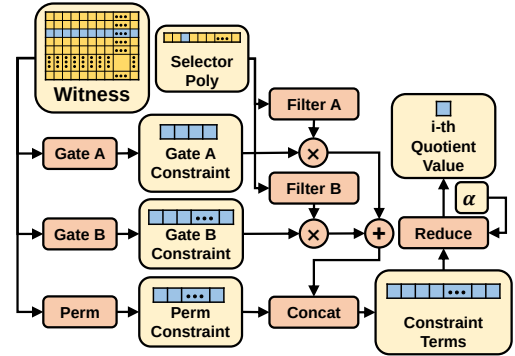


Figure 8: HashMap list and large array structure.

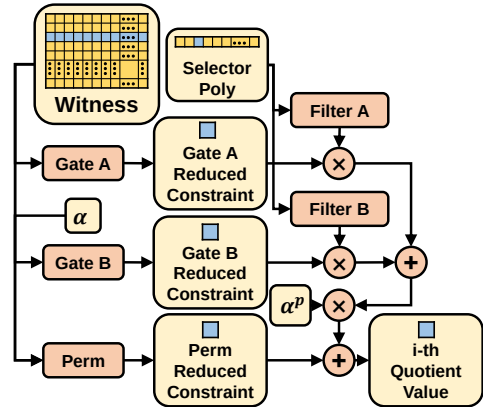
metadata: the gate type (key), the entry length, and its start index in the array. Figure 8(b) demonstrates this process. The large array is then transferred to the GPU. During witness generation, the CPU launches the appropriate GPU kernels, which calculate outputs based on the lookup table and update the witness matrix. The matrix is initialized to zero, with inputs set at the beginning of each proof, ensuring consistency and efficiency across proofs.

3.2.2 In-Place Quotient Polynomials Generation

As shown in Figure 9a. Each value in the quotient polynomials is the reduction of constraint terms with the powers of a quotient challenge α as weights. There are two types of constraint terms in Plonk: gate constraints and permuta-



(a) Original workflow with buffer for gate constraints.



(b) Workflow in ZKPoG.

Figure 9: Quotient generation with new dataflow.

tion constraints. Gate constraints check the input and output ports of the gates to make sure that each gate is executed correctly, and permutation constraints check if the wire topology is satisfied, i.e., if the ports connected by the same wire have the same value. Making use of the feature of Plonkish arithmetization, the wire values of the circuit that are revealed in the proof are constrained by a special type of gate, which eliminates the need for I/O constraints. Since the gate type is related to the position but the form of constraint terms needs to be consistent, one or more selector polynomials are used to combine the terms of different gates.

In the existing CPU implementation, all constraint terms are computed and temporarily stored in a large buffer before undergoing successive filtering, summation, and reduction. The size of this buffer is comparable to the NTT results, as the degree of the quotient polynomials is determined by the circuit’s degree multiplied by a quotient factor (commonly 8), with most circuits containing over 150 constraint terms. This traditional approach results in significant memory overhead and computational redundancy.

To address these inefficiencies, we introduce an innovative approach that leverages the sequential nature of gate constraint computation. As shown in Figure 9b, instead of waiting for all constraint terms to be computed and stored in the buffer, we integrate partial reductions directly during the computation process. The intermediate results are then filtered and accumulated incrementally into the quotient polynomials, significantly reducing memory usage and intermediate data dependencies. For permutation constraints, this method is equally effective, with only an additional power-raising operation required when incorporating the results into the final computation.

By redesigning the computation pipeline, our approach eliminates the need for a buffer to store constraint terms, reducing GPU memory overhead by one-third. Additionally, memory accesses to the buffer are removed, though this introduces additional multiplications during the reduction of gate constraints across different gates. Despite this trade-off, experimental results demonstrate a net improvement in the overall performance of this step.

3.3 FRI PCS

As discussed in 2.2, the commitment phase of FRI contains (inverse-)NTT and Merkle tree building. The acceleration of NTT is not the focus here, so we use the standard optimized Cooley–Tukey algorithm [9] for in-place NTT.

3.3.1 Hash and Merkle Tree

When committing to a set of polynomials, a Merkle tree is built to compress all the values. Assuming there are m polynomials of degree $2^n - 1$ (2^n values). The number of leaf nodes is 2^n , and each leaf node is obtained by compressing the val-

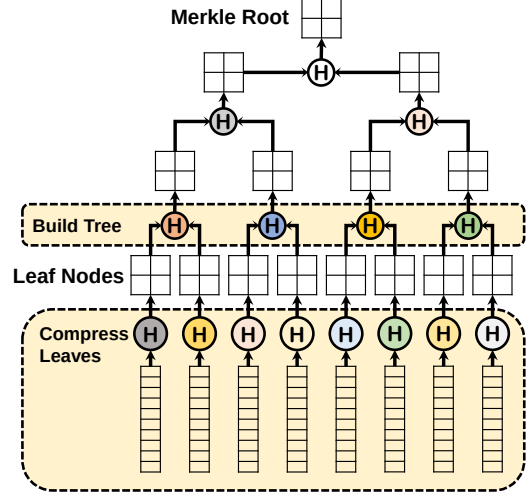


Figure 10: Merkle Tree in FRI. Each hash kernel runs in parallel. Each node has 4 64-bit hash elements.

ues of different polynomials in the same position. As shown in Fig 10, computing the hash function is the only operation in this process. In Plonky2, Poseidon hash [17] is used and can handle 8 values each time, and each leaf node has 4 64-bit values. So $\lfloor m/8 \rfloor \cdot 2^n$ hashes are computed in the leaves step, and around $2^{n-1} + 2^{n-2} + \dots + 1 \approx 2^n$ hashes are computed in the tree step.

Both steps are highly parallelizable, offering significant opportunities for optimization. The 2^n compression procedures in the leaves step are entirely independent, enabling a degree of parallelism that exceeds the number of available GPU threads. Similarly, the Merkle tree can be constructed layer by layer in parallel, with hash computations within the same layer being independent. However, based on our analysis of surveyed applications, the number of polynomials m in the first commitment phase is consistently no less than 135. This results in the leaves generation step dominating the computation overhead, making it the primary bottleneck for acceleration, even when fully parallelized. Experimental results confirm this observation, with Merkle tree building accounting for less than 10% of the total hash function computation time. Consequently, optimization strategies like the pipeline method proposed in BatchZK [29] have minimal impact in our scenario.

3.3.2 In-Place Bit-Reversal Permutation

The result of the in-place (inverse-)NTT³ is inherently in reverse bit order. For instance, in a 32-point NTT, the $(00101)_b$ -th result resides at the $(10100)_b$ -th address. On the other hand, subsequent operations like polynomial evaluation typically

³The entire 2^n -point NTT is performed by splitting the n stages into 2 or 3 sub-stages, where each of the 2^{n-k_i} threads executes an in-place 2^{k_i} -point NTT, with $\sum_i k_i = n$.

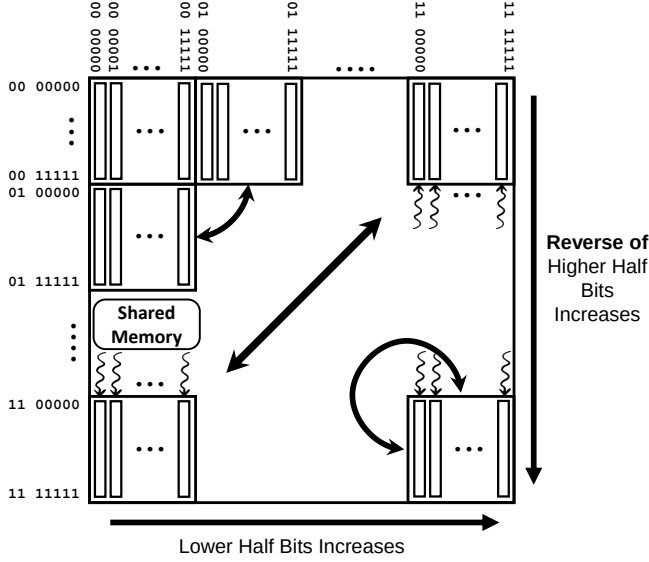


Figure 11: In-place bit-reverse kernel.

require the results in normal order, making direct access too expensive.

Given the substantial size of NTT data—up to 8GB in the first commitment for a degree of 2^{20} —and the constraints of limited GPU memory, we develop an innovative in-place bit-reversal permutation kernel. Our design leverages GPU shared memory and memory coalescing to minimize memory overhead and maximize throughput.

As illustrated in Figure 11, bit-reversal permutation can be viewed as a matrix transpose operation. By splitting the index bits into two halves and sorting them based on the lower half and the reverse of the upper half, the permutation aligns with a block-wise 32×32 matrix transpose. Each block pair (or a block on the diagonal) is assigned to one warp of 32 threads, ensuring data exchange occurs only within paired blocks. This eliminates inter-warp synchronization, significantly improving efficiency. Within each warp, values from the assigned block are loaded into shared memory, reorganized, and written back to their bit-reversed addresses.

Additionally, we optimize memory access by ensuring memory coalescing. Since each row of data is contiguous in memory but columns are not, each thread within a warp handles a column in a block and processes data in a column-major fashion. This strategy ensures that memory operations are aligned and coalesced, resulting in improved performance and efficient GPU utilization.

3.3.3 Final Polynomial Generation

To demonstrate that all committed polynomials evaluate to specified values at verifier-determined points, modern FRI (or deep FRI) transforms this step into a low-degree test. Specifically, proving that a polynomial $F(x)$ has a value v at location

Algorithm 1 Scanning-based Long Division

```

 $acc \leftarrow 0$ 
for  $i = n - 1, n - 2, \dots, 1$  do
     $acc \leftarrow z \cdot acc + a[i]$ 
     $b[i - 1] \leftarrow acc$ 
end for

```

Algorithm 2 Parallelized Division

```

In total  $p$  threads, each handles  $l = n/p$  coefficients
 $id \in \{0, \dots, p - 1\}$ 
 $acc \leftarrow 0$ 
for  $i = l - 1, l - 2, \dots, 0$  do
     $acc \leftarrow z \cdot acc + a[id * l + i]$ 
     $b[id * l + i] \leftarrow acc$ 
end for
syncthreads()
 $acc \leftarrow 0$ 
for  $i = p - 1, p - 2, \dots, id + 1$  do
     $acc \leftarrow z^l \cdot acc + b[i * l]$ 
end for
syncthreads()
for  $i = l - 1, l - 2, \dots, 0$  do
     $acc \leftarrow z \cdot acc$ 
     $b[id * l + i] \leftarrow b[id * l + i] + acc$ 
end for

```

z (i.e., $F(z) = v$) reduces to showing that $F(x) - v$ is divisible by $(x - z)$. Equivalently, the prover must prove that there exists a polynomial $Q(x)$ such that $F(x) - v = (x - z)Q(x)$, where $Q(x)$ has low degree. This $Q(x)$ is referred to as the "final polynomial", and the prover needs to compute it using a long division over the coefficients of $F(x) - v$ and $(x - z)$. Using the scanning-based long division algorithm (Algorithm 1), the division inadvertently becomes a bottleneck when ported from CPU to GPU, as the algorithm has a single-threaded nature.

To optimize this, we observe that the equation $b_{i-1} = \sum_{j=i}^{n-1} z^{j-i} a_j$ holds for $i \in \{1, \dots, n\}$, where $b_n = 0$ due to degree reduction during division. This multiply-and-add structure inspired the development of a parallel algorithm (Algorithm 2). The algorithm divides the coefficients of the polynomial among threads, where each thread processes a portion of the polynomial. By accumulating contributions from higher-degree terms and adding them to its own part, the algorithm significantly accelerates the division step.

4 Custom Gates and Compiler

Computation related to custom gates is often responsible for a significant percentage of the total proof generation time, as we discussed in Section 2.1. Different from most fixed-function kernels we have covered so far, custom gates are

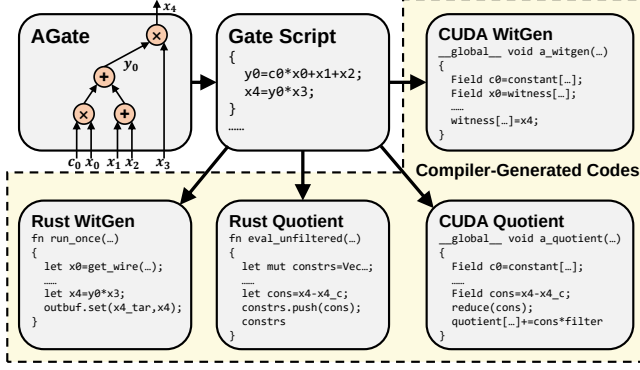


Figure 12: Compiler generated codes based on the gate script.

meant to be the user-customized part of the proof system, and any GPU acceleration must be a compiler-based solution instead of a library-based one. This section looks into how our system automatically derives the necessary code from a small domain-specific language, not only for the GPU kernels but also for the CPU stubs. Even on GPU-less platforms, the user’s experience using custom gates is also improved with the automatic code generation capability.

Custom gates manifest in two computationally intensive steps: 1) the witness generation, where we need to compute the output of the gate from the input, and 2) the quotient polynomial generation, where the difference polynomials are computed between the polynomials containing the inputs, outputs, and the internal values of the gates. The first step takes place on the graph (of the circuit), while the second step takes place on the witness matrix. We derive the necessary code for both steps from a single source: a short user-provided script called the *gate script*. That’s all a user needs to do, and the rest works automatically.

Figure 12 shows the flow of the compiler. The compiler takes the gate script as input, analyzes and translates the variables and operations within, and adds necessary auxiliary code such as memory load-store and constraint term reduction. Based on the gate script, both Rust code for CPUs and CUDA code for GPUs can be generated and added to the current ZKPOG system, which lowers the threshold for migration to other applications.

5 Evaluation

5.1 Methodology

Experimental setup. Our experiments are conducted on a server with two 2.0 GHz Intel(R) Xeon(R) Platinum 8480+ processors (each processor has 56 cores and 112 threads) and 704GB DRAM, running Ubuntu 22.04.5 LTS. It is equipped with an NVIDIA GeForce RTX 4090 GPU with 24GB memory, and we use the CUDA 12.3 toolkit to compile and run the GPU programs.

Baselines. We compare ZKPOG with two baselines: the original Plonky2 (using CPU) and the state-of-the-art Plonky2 CUDA implementation [35]. The GPU baseline only accelerates the NTT, Merkle tree, and elementwise polynomial computations. The other steps such as the witness generation, partial product and quotient are still executed on the CPU.

Applications. We evaluate ZKPOG on various applications to demonstrate its performance improvement over the baselines. The applications include **Factorial**, **Fibonacci**, **SHA256**, **SHA512**, **Ed25519**, **ECDSA**, and **MVM**. The information about the applications is shown in Table 5.

5.2 End-to-End Performance

Table 6 shows the overall performance of ZKPOG and the baselines on different applications. For each application, we present the end-to-end proving time (in seconds) and the ZKPOG speedup over both the CPU and GPU baselines. The results show that ZKPOG significantly outperforms the baselines on all applications, with an average speedup of $22.8\times$ over the CPU baseline.

The MVM application achieves the highest $52.0\times$ speedup over the CPU baseline due to its highly parallel matrix-vector multiplication structure, which fully utilizes GPU resources. The Fibonacci application is an extreme case with only a $3.1\times$ speedup over the CPU baseline due to its highly serial gate structure, limiting GPU acceleration. Witness generation dominates the computation, consuming 68% of the total proving time and bottlenecking overall performance. However, this is an extreme case, as real-world applications typically have more parallelizable circuit structures.

From this we can see that the speedup of witness generation is critical for ZKPOG’s overall performance. To enhance this, we explore various optimization strategies tailored to different circuit structures, which vary in parallelism and computational complexity. We define a metric, *circuit width*, as the ratio of the total number of gates to the number of levels sorted during the build stage, quantifying the parallelism available for GPU execution. Higher *circuit width* indicates greater parallelism and improved performance. Table 7 highlights *circuit width* values across applications.

Based on *circuit width* values and computational complexity, we categorize applications into three types: (1) High parallelism and low complexity applications, such as SHA512 and MVM, leverage low computational demands, enabling GPUs to execute multiple gates simultaneously with minimal overhead. (2) High parallelism but high complexity applications, such as Ed25519, exhibit parallelism but involve expensive operations (e.g., big-number inversions). For these, computationally expensive tasks are offloaded to the CPU to alleviate GPU load. (3) Low parallelism applications, such as Fibonacci, contain highly serial circuits with gate dependencies limiting parallel execution. Here, adjacent-level gates are

Applications	Description	Degree	Gate number	Gate type ⁴
Factorial	The factorial value 1000000!	2^{20}	2000129	1 ~4
Fibonacci	The 2000000-th term of the Fibonacci series	2^{17}	2000134	1 ~4
SHA256	Hash of 60 messages (128B each)	2^{20}	17074766	1 ~3, 5 ~7
SHA512	Hash of 20 messages (128B each)	2^{20}	9917430	1 ~3, 5 ~11
Ed25519	Signature Verification of 2 messages	2^{19}	2009541	1 ~21
ECDSA	Signature Verification of 6 messages	2^{20}	1960939	1 ~3, 6 ~8, 11 ~19, 22
MVM	Matrix-vector multiplication of a 4000x4000 matrix	2^{18}	16132147	1 ~4

Table 5: Information of selected applications.

Applications	Time(s)			Speedup	
	CPU	GPU	ZKPoG	/CPU	/GPU
Factorial	29.95	18.26	1.23	24.4×	14.8×
Fibonacci	3.32	1.67	1.07	3.1×	1.6×
SHA256	41.87	21.66	2.31	18.1×	9.4×
SHA512	33.82	23.24	1.58	21.4×	14.7×
Ed25519	23.30	20.04	1.51	15.5×	13.3×
ECDSA	34.73	21.86	1.37	25.3×	15.9×
MVM	30.74	11.14	0.59	52.0×	18.9×

Table 6: Performance of ZKPoG on different applications.

Application	Circuit width	Proportion of WitGen
Factorial	2.0	47.7%
Fibonacci	1.0	92.1%
SHA256	5918.5	67.3%
SHA512	3868.0	54.6%
Ed25519	633.9	61.1%
ECDSA	1177.0	46.1%
MVM	3584.1	16.4%

Table 7: The circuit width and the proportion of time spent in witness generation of different applications.

consolidated to reduce GPU context switching overhead.

Figure 13 shows the breakdown of the proving time for each application. For most steps of the proving process, ZKPoG achieves about $25.7 \times$ speedup over the CPU baseline, except for the witness generation step, which only achieves a $14.8 \times$ speedup. In addition, the witness generation step takes up 58% of the total proving time, which has now become the bottleneck of the proving process.

Compared to the GPU baseline, ZKPoG achieves an average speedup of $12.7 \times$ across all applications. While the GPU baseline accelerates some steps (e.g., commitment), a signifi-

⁴The number in the gate type column corresponding to the following gates: RandomValue, Constant, ArithmeticBase, Poseidon, BaseSum, BaseSplit, U32Arithmetic, U32Subtraction, U32Interleave, UninterleaveToU32, U32AddMany, U32RangeCheck, Equality, Comparison, RandomAccess, NonNativeAddition, NonNativeSubtraction, NonNativeMultiplication, NonNativeInverse, BigUIntDivRem, CurvePointDecompression, GLVDecomposition.

Step	GPU Time(ms)	ZKPoG Time(ms)	Speedup
Commit1	1746.4	330.9	$5.28 \times$
Commit2	698.4	66.3	$10.53 \times$
Commit3	617.8	50.0	$12.36 \times$

Table 8: Comparison of kernel speedup between baseline GPU and ZKPoG.

cant portion, especially witness generation, still relies on the CPU, limiting overall performance. In contrast, ZKPoG accelerates the entire proving process, delivering substantial improvements. Additionally, ZKPoG achieves higher speedups even for GPU-accelerated computations, as shown in Table 8, highlighting commit1, commit2, and commit3 from Figure 7.

5.3 Memory Optimizations and Applications

We compared the peak memory usage of ZKPoG on different applications before and after memory optimization. The results are shown in Figure 14. The memory usage of ZKPoG on all applications is reduced by 62.7% on average after memory optimization. In addition, Figure 14 shows the maximum degree of different applications that can run on our limited-memory (24GB) GPU before and after the memory optimization. The maximum degree of all applications is increased by 1.57 on average after memory optimization. The circuit scale grows exponentially with degree. Therefore, memory optimization is crucial for running large-scale circuits on limited-memory GPUs.

5.4 Custom Gate Performance

We use two circuits with relatively simple structures, Factorial and Fibonacci, to test the performance of custom gate. Factorial circuit includes many multiplication gates in series, so a custom gate that does multiple multiplications can be used. Factorial circuit also contains a ladder of addition gates, so a fixed number of steps on the ladder can be replaced by a single custom gate. As shown in Table 9, with custom gates,

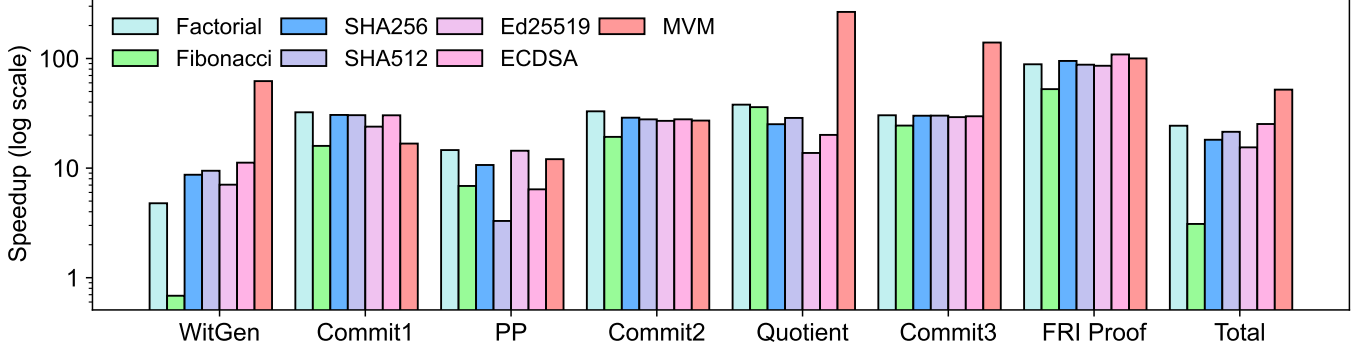


Figure 13: Speedup of each step in ZKPoG over the CPU baseline.

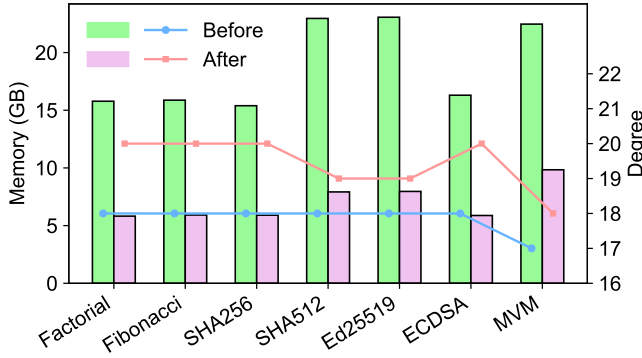


Figure 14: The peak memory usage and maximum degree of ZKPoG before and after memory optimization. The bar graph represents memory usage and the line graph represents degree.

Time(s)	CPU	Ours	CPU	Ours
Degree			+ Custom	+ Custom
Factorial	29.7 2^{20}	1.24 2^{20}	2.64 2^{17}	0.20 2^{17}
Fibonacci	3.21 2^{17}	1.07 2^{17}	0.08 2^{11}	0.03 2^{11}

Table 9: Performance of custom gates.

the circuit size (measured in terms of the degree of the polynomial) decreases significantly, as does the prover time. Due to the interface we defined and the flexibility of GPU programs, ZKPoG still achieves a high speedup ratio on the basis of custom gates.

5.5 Configuration Parameters

Figure 15 shows how the performance of ZKPoG changes with the circuit scale. We use Factorial as an example and increase its order to control the degree of the circuit. Each dotted line in the graph represents where the degree changes.

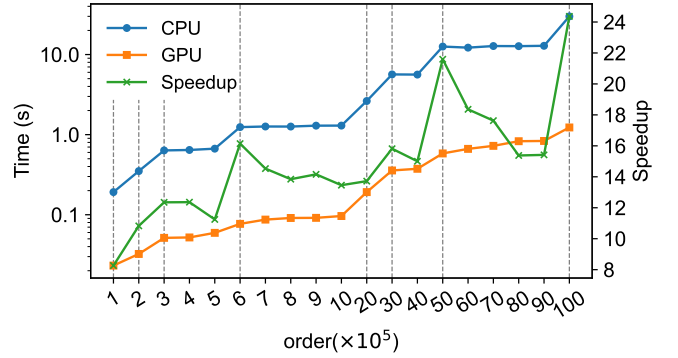


Figure 15: Performance of circuit in different scales.

The results show that the proving time of both the CPU baseline and ZKPoG increases exponentially with the degree of the circuit, but the growth rate of ZKPoG is much slower than that of the CPU. As a result, the speedup of ZKPoG over the CPU increases with the degree of the circuit. At the same degree, the speedup decreases slightly as the order increases, since the witness generation step becomes more time-consuming, while other steps remain the same.

6 Conclusion

In this paper, we present ZKPoG, a witness generation incorporated end-to-end zero-knowledge proof acceleration platform based on GPU. ZKPoG accelerates the long-neglected frontend of the zk-SNARK prover by parallelizing gate execution according to their position and dependencies. Additionally, it leverages in-place operations to save device memory and is enhanced by a compiler that generated GPU and CPU codes based on the gate description, making it more versatile. Compared to the parallelized CPU implementation and an open-source GPU implementation, ZKPoG achieves a substantial speedup on different applications.

References

- [1] S. Ames, C. Hazay, Y. Ishai, and M. Venkatasubramanian, “Ligero: Lightweight sublinear arguments without a trusted setup,” Cryptology ePrint Archive, Paper 2022/1608, 2022. [Online]. Available: <https://eprint.iacr.org/2022/1608>
- [2] J. Bell, A. Gascón, T. Lepoint, B. Li, S. Meiklejohn, M. Raykova, and C. Yun, “{ACORN}: input validation for secure aggregation,” in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 4805–4822.
- [3] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Fast Reed-Solomon Interactive Oracle Proofs of Proximity,” in *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), I. Chatzigiannakis, C. Kaklamanis, D. Marx, and D. Sannella, Eds., vol. 107. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018, pp. 14:1–14:17. [Online]. Available: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ICALP.2018.14>
- [4] E. Ben-Sasson, A. Chiesa, D. Genkin, E. Tromer, and M. Virza, “Succinct non-interactive zero knowledge for a von neumann architecture,” in *23rd USENIX Security Symposium (USENIX Security 14)*, 2014, pp. 781–796.
- [5] E. Ben-Sasson, A. Chiesa, M. Riabzev, N. Spooner, M. Virza, and N. P. Ward, “Aurora: Transparent succinct arguments for r1cs,” vol. 11476, pp. 103–128, 2019.
- [6] M. Blum, P. Feldman, and S. Micali, “Non-interactive zero-knowledge and its applications,” in *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing*, ser. STOC ’88. New York, NY, USA: Association for Computing Machinery, 1988, p. 103–112. [Online]. Available: <https://doi.org/10.1145/62212.62222>
- [7] S. Bowe, J. Grigg, and D. Hopwood, “Recursive proof composition without a trusted setup,” Cryptology ePrint Archive, Paper 2019/1021, 2019. [Online]. Available: <https://eprint.iacr.org/2019/1021>
- [8] B. Chen, B. Bünz, D. Boneh, and Z. Zhang, “HyperPlonk: Plonk with linear-time prover and high-degree custom gates,” Cryptology ePrint Archive, Paper 2022/1355, 2022. [Online]. Available: <https://eprint.iacr.org/2022/1355>
- [9] J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex fourier series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965. [Online]. Available: <http://www.jstor.org/stable/2003354>
- [10] A. Daftardar, B. Reagen, and S. Garg, “Szkp: A scalable accelerator architecture for zero-knowledge proofs,” in *Proceedings of the 2024 International Conference on Parallel Architectures and Compilation Techniques*, ser. PACT ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 271–283. [Online]. Available: <https://doi.org/10.1145/3656019.3676898>
- [11] A. Fiat and A. Shamir, “How to prove yourself: practical solutions to identification and signature problems,” in *Proceedings on Advances in Cryptology—CRYPTO ’86*. Berlin, Heidelberg: Springer-Verlag, 1987, p. 186–194.
- [12] A. Gabizon and Z. J. Williamson, “plookup: A simplified polynomial protocol for lookup tables,” Cryptology ePrint Archive, Paper 2020/315, 2020. [Online]. Available: <https://eprint.iacr.org/2020/315>
- [13] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, “PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge,” Cryptology ePrint Archive, Paper 2019/953, 2019. [Online]. Available: <https://eprint.iacr.org/2019/953>
- [14] L. Goldberg and A. Smith, “A performance study of zk-snarks: Achieving scalability in private blockchains,” *IEEE Transactions on Blockchain*, vol. 2, no. 1, pp. 45–60, 2021.
- [15] S. Goldwasser, S. Micali, and C. Rackoff, “The knowledge complexity of interactive proof-systems,” *SIAM Journal on Computing*, 1985.
- [16] A. Golovnev, J. Lee, S. Setty, J. Thaler, and R. S. Wahby, “Brakedown: Linear-time and field-agnostic snarks for r1cs,” in *Advances in Cryptology – CRYPTO 2023*, H. Handschuh and A. Lysyanskaya, Eds. Cham: Springer Nature Switzerland, 2023, pp. 193–226.
- [17] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schafneggner, “Poseidon: A new hash function for zero-knowledge proof systems,” Cryptology ePrint Archive, Paper 2019/458, 2019. [Online]. Available: <https://eprint.iacr.org/2019/458>
- [18] J. Groth, “On the size of pairing-based non-interactive arguments,” *Advances in Cryptology—EUROCRYPT 2016*, 2016.
- [19] K. group, “Opencl: Open standard for parallel programming of heterogeneous systems,” 2024. [Online]. Available: <https://www.khronos.org/opencl/>
- [20] —, “Opengl: The industry’s foundation for high performance graphics,” 2024. [Online]. Available: <https://www.opengl.org/>

- [21] P. Gupta, “Cuda refresher: The cuda programming model,” 2020. [Online]. Available: <https://developer.nvidia.com/blog/cuda-refresher-cuda-programming-model/>
- [22] H. Huang, P. Zhu, F. Xiao, X. Sun, and Q. Huang, “A blockchain-based scheme for privacy-preserving and secure sharing of medical data,” *Computers & Security*, vol. 99, p. 102010, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404820302832>
- [23] ingyama. (2025) Icycle. [Online]. Available: <https://github.com/ingonyama-zk/icicle>
- [24] Y. Ishai, E. Kushilevitz, R. Ostrovsky, and A. Sahai, “Zero-knowledge from secure multiparty computation,” in *Proceedings of the thirty-ninth annual ACM symposium on Theory of computing*, 2007, pp. 21–30.
- [25] Z. Ji, Z. Zhang, J. Xu, and L. Ju, “Accelerating multi-scalar multiplication for efficient zero knowledge proofs with multi-gpu systems,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 57–70. [Online]. Available: <https://doi.org/10.1145/3620666.3651364>
- [26] A. Kate, G. M. Zaverucha, and I. Goldberg, “Constant-size commitments to polynomials and their applications,” in *Advances in Cryptology - ASIACRYPT 2010 - 16th International Conference on the Theory and Application of Cryptology and Information Security*, ser. Lecture Notes in Computer Science, vol. 6477. Springer, 2010, pp. 177–194. [Online]. Available: <https://www.iacr.org/archive/asiacrypt2010/6477178/6477178.pdf>
- [27] A. Kosba, A. Miller, E. Shi, Z. Wen, and C. Papamanthou, “Hawk: The blockchain model of cryptography and privacy-preserving smart contracts,” *2016 IEEE Symposium on Security and Privacy (SP)*, pp. 839–858, 2016.
- [28] P. labs. (2025) Plonky2. [Online]. Available: <https://github.com/0xPolygonZero/plonky2>
- [29] T. Lu, Y. Chen, Z. Wang, X. Wang, W. Chen, and J. Zhang, “BatchZK: A fully pipelined GPU-accelerated system for batch generation of zero-knowledge proofs,” *Cryptology ePrint Archive*, Paper 2024/1862, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1862>
- [30] T. Lu, C. Wei, R. Yu, C. Chen, W. Fang, L. Wang, Z. Wang, and W. Chen, “cuZK: Accelerating zero-knowledge proof with a faster parallel multi-scalar multiplication algorithm on GPUs,” *Cryptology ePrint Archive*, Paper 2022/1321, 2022. [Online]. Available: <https://eprint.iacr.org/2022/1321>
- [31] W. Ma, Q. Xiong, X. Shi, X. Ma, H. Jin, H. Kuang, M. Gao, Y. Zhang, H. Shen, and W. Hu, “Gzpk: A gpu accelerated zero-knowledge proof system,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 340–353. [Online]. Available: <https://doi.org/10.1145/3575693.3575711>
- [32] N. Narula, W. Vasquez, and M. Virza, “zkledger: privacy-preserving auditing for distributed ledgers,” in *Proceedings of the 15th USENIX Conference on Networked Systems Design and Implementation*, ser. NSDI’18. USA: USENIX Association, 2018, p. 65–80.
- [33] Nvidia, “Cuda toolkit,” 2024. [Online]. Available: <https://developer.nvidia.com/cuda-toolkit>
- [34] OpenACC, “Openacc-standard.org,” 2024. [Online]. Available: <https://www.openacc.org/>
- [35] S. Protocol, “plonky2-gpu,” 2024. [Online]. Available: <https://github.com/sideprotocol/plonky2-gpu/>
- [36] M. Richardson, *Blockchain Technology: Learn Step by Step Smart Contracts, Monero, Blockchain Wallet, Bitcoin, Zcash, Ethereum, Ripple, Dash, Mining, Litecoin, IOTA*. Mike Richardson, 2024.
- [37] A. Roy Chowdhury, C. Guo, S. Jha, and L. van der Maaten, “Eiffel: Ensuring integrity for federated learning,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 2535–2549.
- [38] N. Samardzic, S. Langowski, S. Devadas, and D. Sanchez, “Accelerating Zero-Knowledge Proofs Through Hardware-Algorithm Co-Design,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. Los Alamitos, CA, USA: IEEE Computer Society, Nov. 2024, pp. 366–379. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/MICRO61859.2024.00035>
- [39] Scroll. [Online]. Available: <https://scroll.io>
- [40] S. Setty, “Spartan: Efficient and general-purpose zkSNARKs without trusted setup,” *Cryptology ePrint Archive*, Paper 2019/550, 2019. [Online]. Available: <https://eprint.iacr.org/2019/550>
- [41] T. Xie, Y. Zhang, and D. Song, “Orion: Zero knowledge proof with linear prover time,” *Cryptology ePrint*

Archive, Paper 2022/1010, 2022. [Online]. Available: <https://eprint.iacr.org/2022/1010>

- [42] Zcash. (2025) Halo2. [Online]. Available: <https://github.com/zcash/halo2>
- [43] Y. Zhang, S. Wang, X. Zhang, J. Dong, X. Mao, F. Long, C. Wang, D. Zhou, M. Gao, and G. Sun, “Pipezk: Accelerating zero-knowledge proof with a pipelined architecture,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 416–428.
- [44] zkSync. [Online]. Available: <https://zksync.io>